

贪心算法

贪心算法

贪心算法（又称贪婪算法）是指，在对问题求解时，总是做出在当前看来是最好的选择。也就是说，不从整体最优上加以考虑，他所做出的仅是在某种意义上的局部最优解

贪心算法不是对所有问题都能得到整体最优解，关键是贪心策略的选择，选择的贪心策略必须具备无后效性，即某个状态以前的过程不会影响以后的状态，只与当前状态有关。

贪心算法的特性

贪婪算法可解决的问题通常大部分都有如下的特性：

- (1)随着算法的进行，将积累起其它两个集合：一个包含已经被考虑过并被选出的候选对象，另一个包含已经被考虑过但被丢弃的候选对象。
 - (2)有一个函数来检查一个候选对象的集合是否提供了问题的解答。该函数不考虑此时的解决方法是否最优。
 - (3)还有一个函数检查是否一个候选对象的集合是可行的，也即是否可能往该集合上添加更多的候选对象以获得一个解。和上一个函数一样，此时不考虑解决方法的最优性。
 - (4)选择函数可以指出哪一个剩余的候选对象最有希望构成问题的解。
 - (5)最后，目标函数给出解的值。
 - (6)为了解决问题，需要寻找一个构成解的候选对象集合，它可以优化目标函数，贪婪算法一步一步的进行。起初，算法选出的候选对象的集合为空。接下来的每一步中，根据选择函数，算法从剩余候选对象中选出最有希望构成解的对象。如果集合中加上该对象后不可行，那么该对象就被丢弃并不再考虑；否则就加到集合里。每一次都扩充集合，并检查该集合是否构成解。如果贪婪算法正确工作，那么找到的第一个解通常是最优的。
-

基本思路

1. 建立数学模型来描述问题
2. 把求解的问题分成若干个子问题。
3. 对每一子问题求解，得到子问题的局部最优解。
4. 把子问题的解局部最优解合成原来解问题的一个解。

实现该算法的过程：

从问题的某一初始解出发；

while 能朝给定总目标前进一步

do

求出可行解的一个解元素；

由所有解元素组合成问题的一个可行解。

下面是一个可以试用贪心算法解的题目，贪心解的确不错，可惜不是最优解。

例题分析

[0-1背包问题]有一个背包，背包容量是 $M=150$ 有7个物品，物品不可以分割成任意大小。
要求尽可能让装入背包中的物品总价值最大，但不能超过总容量。

物品 A B C D E F G

重量 35kg 30kg 6kg 50kg 40kg 10kg 25kg

价值 10\$ 40\$ 30\$ 50\$ 35\$ 40\$ 30\$

例题分析

分析：

目标函数： $\sum p_i$ 最大

约束条件是装入的物品总重量不超过背包容量： $\sum w_i \leq M (M=150)$

(1)根据贪心的策略，每次挑选价值最大的物品装入背包，得到的结果是否最优？

(2)每次挑选所占重量最小的物品装入是否能得到最优解？

(3)每次选取单位重量价值最大的物品，成为解本题的策略。

值得注意的是，贪心算法并不是完全不可以使用，贪心策略一旦经过证明成立后，它就是一种高效的算法。

贪心算法还是很常见的算法之一，这是由于它简单易行，构造贪心策略不是很困难。

可惜的是，它需要证明后才能真正运用到题目的算法中。

一般来说，贪心算法的证明围绕着：整个问题的最优解一定由在贪心策略中存在的子问题的最优解得来的。

对于例题中的3种贪心策略，都是无法成立（无法被证明）的

例题分析

(1)贪心策略：选取价值最大者。

反例：

$W=30$

物品：A B C

重量：28 12 12

价值：30 20 20

根据策略，首先选取物品A，接下来就无法再选取了，可是，选取B、C则更好。

(2)贪心策略：选取重量最小。它的反例与第一种策略的反例差不多。

(3)贪心策略：选取单位重量价值最大的物品。

反例：

$W=30$

物品：A B C

重量：28 20 10

价值：28 20 10

根据策略，三种物品单位重量价值一样，程序无法依据现有策略作出判断，如果选择A，则答案错误。

【注意：如果物品可以分割为任意大小，那么策略3可得最优解】
本题是个DP问题，用贪心法并不一定可以求得最优解

例题2

马的遍历问题。在 8×8 方格的棋盘上，从任意指定方格出发，为马寻找一条走遍棋盘每一格并且只经过一次的一条最短路径。

问题分析：

首先这是一个搜索问题，运用深度优先搜索进行求解。

这样做是完全可行的，它输入的是全部解，但是马遍历当 8×8 时解是非常之多的，用天文数字形容也不为过，这样一来求解的过程就非常慢，并且出一个解也非常慢。

是否有跟好的方法呢？

贪心算法

其实马踏棋盘的问题很早就有人提出，且早在1823年，J.C.Warnsdorff就提出了一个有名的算法。在每个结点对其子结点进行选取时，优先选择‘出口’最小的进行搜索，‘出口’的意思是在这些子结点中它们的可行子结点的个数，也就是‘孙子’结点越少的越优先跳，为什么要这样选取，这是一种局部调整最优的做法，如果优先选择出口多的子结点，那出口少的子结点就会越来越多，很可能出现‘死’结点（顾名思义就是没有出口又没有跳过的结点），这样对下面的搜索纯粹是徒劳，这样会浪费很多无用的时间，反过来如果每次都优先选择出口少的结点跳，那出口少的结点就会越来越少，这样跳成功的机会就更大一些。

例题3:

在 N 行 M 列的正整数矩阵中，要求从每行中选出1个数，使得宣传的总共 N 个数和最大

排队打水问题

有 n 个人排队到 r 个水龙头去打水，他们装满水桶的时间为 $t_1, t_2, \dots, t_n$ 为整数，且各个不相等，应如何安排打水顺序才能使他们花费的时间最少呢？

问题分析：由于排队时，越靠前面的计算次数越多，因此越小的排在越前面得出的结构越小

纪念品分组

元旦快到了，校学生会让乐乐负责新年晚会的纪念品发放工作，为使得参加晚会的同学说获得的纪念品价值相对均衡，他要把够来的纪念品根据价值进行分组，但每组最多只能包括两件纪念品，并且每组纪念品的价格之和不能超过一个给定的整数。为了保证在尽量短的时间内发完所有纪念品，乐乐希望分组的数目最少？

你的任务写一个程序，找出所有分组方案中分组数最少的一中，输出最少的分组数目。

输入文件**group.in** 包含 **$n+2$** 行

第一行包括一个整数 **w** 为每组纪念品价格之和的上限

第**2**行为一个整数 **n** ，表示购来的纪念品的总件数

第**3**至 **$n+2$** 行每行包含一个正整数 **p_i** ($5 \leq p_i \leq w$)，
表示所对应纪念品的价格

输出文件**group.out** 仅一行，包含一个整数，即最少的
分组数目。

样例输入：

100

9

90

20

20

30

50

60

70

80

90

样例输出：

6

数据范围：

50%数据： $1 \leq n \leq 15$;

100%数据满足： $1 \leq n \leq 30000, 80 \leq w \leq 200$
